

Redrob Hackathon - Gap Analysis & Structured Plan

Project audited: Lumina-Intelligent-Hiring (Red-Robe-Lumina)

Challenge:

1. Headline Finding - Your project solves a different problem

Your repo (Lumina-Intelligent-Hiring) is a full-stack AI hiring-assessment & proctoring platform:

- React frontend (src/) + Node/Express/MongoDB backend (backend/) + Python FastAPI "AI engine" (main.py, core/).

- It parse
shows lea

The hackathon asks for something almost entirely different: a single, offline, CPU-only batch ranker that reads candidates.jsonl (100K records) and emits a top-100 submission.csv in ≤ 5 minutes with no network / no LLM calls.

These overlap only in theme ("hiring AI"). Almost none of the current code is usable for the submission, and its core design (per-request cloud LLM calls) is explicitly forbidden by the rules. Treat this as a fresh build, not a patch.

2. What the Challenge Requires (distilled from the spec)

#	Requirement	Source
R1	rank.py reads candidates.jsonl -> writes top-100 submission.csv	submission_spec sec.2, metadata template
R2	CSV: header + exactly 100 rows; cols candidate_id,rank,score,reasoning; ranks 1-100 unique; score non-increasing; ties broken by candidate_id ascending	spec sec.2-3, validate_submission.py
R3	≤ 5 min, ≤ 16 GB RAM, CPU-only, network OFF, ≤ 5 GB disk during ranking	spec sec.3
R4	Ranking must reflect JD meaning, not keywords; avoid keyword-stuffer traps	job_description (final note)
R5	Avoid ~80 honeypots (impossible profiles) - >10% in top 100 = disqualified	spec sec.7
R6	Weight 23 behavioral signals (response rate, last_active, etc.) as a modifier	redrob_signals_doc
R7	Per-candidate reasoning: specific, JD-connected, honest, non-templated, no hallucination	spec sec.3 (Stage 4)
R8	Optimize for NDCG@10 (0.5), NDCG@50 (0.3), MAP (0.15), P@10 (0.05)	spec sec.4
R9	GitHub repo: README w/ single reproduce command, requirements.txt, pre-compute scripts, submission_metadata.yaml	spec sec.10.3
R10	Working hosted sandbox (HF Spaces / Streamlit / Colab / Docker / Binder)	spec sec.10.5
R11	submission_metadata.yaml filled (team, GitHub, sandbox, compute env, AI-tools declaration, methodology)	template
R12	Honest git history showing real iteration (not one dump); defensible in interview	spec sec.5 (Stage 4-5)

3. Gap Analysis - what's missing vs. what exists

Needed	In your repo today?	Verdict
rank.py batch ranker (CPU/offline)	None	Missing - core deliverable
candidates.jsonl loader (stream 100K, low-mem)	None (current ingests PDF/text via API)	Missing

Needed	In your repo today?	Verdict
JD-vs-candidate scoring without LLM API (local embeddings/TF-IDF + rules)	Logic exists but depends on Groq/Ollama calls -> violates R3	Rebuild required
Title/role gate (reject "Marketing Manager w/ AI keywords")	None	Missing - high value (NDCG@10)
Honeypot / impossible-profile detector	None	Missing - DQ risk
Behavioral-signal modifier (23 signals)	None	Missing
Reasoning generator (offline, fact-grounded)	Only LLM-based, non-grounded	Rebuild required
submission.csv writer + tie-break + validator pass	None	Missing
requirements.txt scoped to ranker	Exists but for FastAPI/web stack	Replace
submission_metadata.yaml	Not created	Missing (R11)
README with single reproduce command	2-line README	Rewrite (R9)
Hosted sandbox	Has Vercel/Render web deploy, not a ranker sandbox	Missing (R10)
Local NDCG/MAP eval harness (self-validation)	None	Missing - needed since no leaderboard
Meaningful git history	Single "Initial commit"	Build via real iteration (R12)

Reusable from current repo: essentially only ideas - the skill-normalization vocabulary and JD-skill-matching heuristics in `core/jd_parser.py` / `core/resume_parser.py` can inform feature design. The code itself can't be reused because it routes through cloud LLMs. Copy in `validate_submission.py` and run it on every output.

4. The 5 things that make or break the score

1. JD meaning over keywords - score role/title fit and real career evidence; a "Marketing Manager" with AI skills is a trap, not a match.

2. Honey
disqualifie

5. Recommended Structure

Build the ranker as a clean, self-contained project (own folder/repo) rather than entangling it with the web app - the spec's Stage-3 Docker reproduction expects a minimal, dependency-pinned ranker, and your web stack (Mongo, React, FastAPI) would only get in the way. The existing platform can stay as a separate showcase.

redrob-ranker/

rank.py

6. Structured Phased Plan (~8-11 days solo)

Phase 0 - Setup (1/2 day)

- Create n

Phase 1 - Data & JD modeling (1 day)

- Streami

Phase 2 - Feature & scoring engine (2-3 days)

- Compon
skills wi
(Pune/No

Phase 3 - Honeypots & behavioral signals (1-2 days)

- Honeyp
bottom (F

Phase 4 - Reasoning + output (1 day)

- Genera
concerns

Phase 5 - Local evaluation (1 day, critical - no leaderboard)

- Build a
check ho

Phase 6 - Performance hardening (1/2-1 day)

- Ensure
(precomp

Phase 7 - Packaging & deliverables (1 day)

- READM

7. Deliverables Checklist (don't miss any)

[] rank.py (single command: `python rank.py --candidates ./candidates.jsonl --out ./submission.csv`)

[] su
non-incre

8. Scoring Formula (for reference)

Final composite = $0.50 \times \text{NDCG@10} + 0.30 \times \text{NDCG@50} + 0.15 \times \text{MAP} + 0.05 \times \text{P@10}$

Tiebreaks between submissions: higher P@5, then higher P@10, then earlier timestamp.

Relevant =